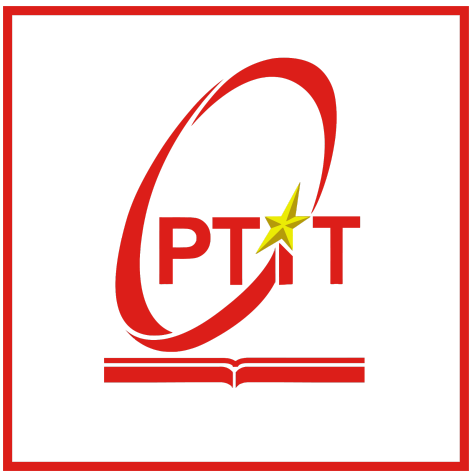
POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 1



PROJECT

ANALYSIS AND DESIGN OF INFORMATION SYSTEMS

Subject: No.16

Class	:	E22CQCN02-B (E22HTTT)
Lecturer	:	Đỗ Thị Bích Ngọc
Author	:	Nguyễn Minh Khiêm - B22DCDT170
Class group	:	05

Hanoi, 2025

Subject No. 16

Duration: 90 minutes

A cinema management system could be used by management staff, saler and customers:

- Management staff: view the types of statistics: movies, customers and revenue. Schedule showtimes, manage movie and theater information (add, edit, delete).
- Saler: selling tickets at the counter to customers, issuing membership cards to customers
- Customer: register for membership, search, buy tickets online, buy tickets at the counter

Considering two modules and answering questions

- ***Customer registers as a member***: chooses to register as a member → enters personal information and clicks register → the system saves information to the database.
- ***Management staff views movie statistics***: selects the menu to view statistical reports → chooses to view movie statistics by revenue → selects start and end date → Views movie statistics → clicks on a movie to view details → views statistics of movie showtimes → clicks on a showtime → views statistics of ticket sales of the showtime.

Question 1 (2 points)

a. Build the use case diagram for the two modules

b. Present scenarios for the two modules

Question 2 (2 points)

a. Extract the entity classes (class name, attributes) related to the two modules

b. Build the entity class diagram for the extracted classes.

Question 3 (2 points)

a. Build the communication diagram for the standard scenario of the two modules.

b. Build the detail design class diagram for the two modules.

Question 4 (2 points)

a. Based on the entity classes of Question 2, design the database related to the two modules

b. Generate Java code (class template, declaration of attributes and methods, explain how methods work) for classes in the diagram of Question 3.b.

Question 5 (2 points)

a. Build the package diagram for classes in the diagram of Question 3.b.

b. Build the deployment diagram for MVC model based on the J2EE technology.

A. Requirements

1. Build the glossary list:

No	Vietnamese Term	English Term	Meaning
Group: Human-related concepts			
1	Nhân viên quản lý	Management staff	Staff members responsible for managing and supervising the entire cinema's operations. They have full access to system functions, including viewing statistics (movies, customers, revenue), scheduling showtimes, and managing movie/theater information (add, edit, delete).
2	Nhân viên bán vé	Saler	People who directly sell tickets at the counter, issue membership cards.
3	Khách hàng	Customer	People who buy tickets and use cinema services. A Customer may be unregistered (buy Tickets at the counter) or registered (Member). A registered Customer has personal information stored in the Database, can receive a Membership card, buy Tickets online, get promotions, and be included in statistical reports.
4	Thành viên	Member	A Customer who has registered in the system. Members provide personal information stored in the Database and can access more functions, such as buying Tickets online, receiving promotions, and being included in membership management and statistics.
Group: Human activity-related concepts			
5	Xem thống kê	View statistics	An activity performed by Management staff to access Statistical reports and monitor business performance. Reports may include Revenue, sold Tickets, customer data, and performance by Movie or Showtime. The results are used to evaluate trends, identify effective showtimes/movies, and adjust business plans.
6	Lập lịch chiếu	Schedule showtimes	The process of arranging Showtime for each Movie in suitable time slots within a Theater. The goal is to maximize customer attendance, avoid unnecessary overlaps, ensure continuous schedules, and generate accurate data for ticket sales and reports.
7	Quản lý thông tin	Manage movie	Adding, editing, or deleting Movie data (title,

	phim	information (add, edit, delete)	genre, duration, description, displayed ticket price). Ensures consistent information so users can search and purchase without confusion. All updates are saved in the Database.
8	Bán vé tại quầy	Selling tickets at the counter	A process where the Saler selects a Movie and Showtime, generates a Ticket, collects payment, and provides the printed ticket to the Customer. The system updates the available seats for the Showtime and records the transaction into the Database.
9	Quản lý thông tin rạp	Manage theater information (add, edit, delete)	Updating Theater data (name, identifying details, operational status). Accurate theater information is required for linking showtimes correctly and for generating reliable reports by cinema location.
10	Phát hành thẻ thành viên	Issuing membership cards	The activity of creating and providing a Membership card for a Customer who has registered. The card is linked with the customer's profile in the Database and used to apply promotions, track loyalty, and include customer data in reports.
11	Đăng ký thành viên	Register for membership	A process where a Customer chooses to register in the system, enters personal information, and confirms. After clicking register, the system saves data into the Database and marks the customer as registered. Registration enables online ticketing, promotions, and inclusion in membership reports.
12	Tìm kiếm phim	Search movies	The action of Customer searching for a Movie based on title, genre, or schedule. The results help customers decide which film to watch and navigate to Showtime selection. It is also used by Saler at the counter to assist customers.
13	Mua vé online	Buy tickets online	A process where a Customer uses the website/app to select a Movie, choose a Showtime, confirm purchase, and complete payment. The system records the transaction, updates ticket availability for the showtime, and generates a valid Ticket for later verification at the cinema.
14	Mua vé tại quầy	Buy tickets at the counter	The traditional way where Customer purchases directly from the Saler. This applies to unregistered customers or those who prefer on-site payment. The transaction is entered into the Database to ensure sales data aligns with online

			channels.
15	Nhập thông tin cá nhân	Enter personal information	The step where a customer provides necessary personal details during registration. The system validates formats and saves the information into the Database for identification, Membership card issuance, and reporting.
16	Nhấn nút đăng ký	Click register	The confirmation step to finalize registration for membership. This triggers the system to create a customer profile, mark the status as registered, and enable all features exclusive to registered customers.
17	Chọn menu và báo cáo	Select menu → choose report	Management staff navigate the Menu to select the type of Statistical report they need. This step specifies the report focus before applying filters like time range.
18	Chọn ngày bắt đầu và kết thúc	Select start and end date	Defines the reporting time range. The system uses this range to compile figures such as Revenue, sold Tickets, and other metrics, ensuring the report reflects the correct analysis period.
19	Chọn phim/suất chiếu	Click on a movie/showtime	A drill-down action from summary reports into detailed data of a specific Movie or Showtime. This allows managers to analyze ticket sales and revenue for a particular item and make decisions about scheduling or marketing.
Group: Object-related concepts			
20	Phim	Movie	The core business object of the cinema. Each Movie has attributes and is linked to multiple Showtime. Ticket sales and Revenue by movie are critical data in statistical reports.
21	Rạp chiếu	Theater	The physical cinema location where films are shown. A Theater contains multiple Showtime, and data is aggregated per theater for operational and financial reporting.
22	Suất chiếu	Showtime	A specific time slot when a Movie is screened at a Theater. Showtime is the basic unit for ticket sales and reporting. Ticket sales and revenue per showtime help managers optimize schedules.
23	Vé	Ticket	The document that grants a Customer the right to attend a Showtime of a Movie at a Theater. Each Ticket is linked to a transaction, stored in the

			Database, and used in sales and revenue statistics.
24	Thẻ thành viên	Membership card	A physical or digital identifier for a registered Customer. It is linked to the customer's profile in the Database, supports discounts and promotions, tracks loyalty, and is included in membership statistics.
25	Thông tin khách hàng	Customer information	The stored profile of a Customer in the Database, including essential personal details. It is required for registration, Membership card issuance, online purchases, and customer-based reports.
26	Cơ sở dữ liệu	Database	The central repository stores all data of the system: Movie, Theater, Showtime, Ticket, Customer, Membership card, Revenue, and reports. Ensures data integrity, supports both operational and analytical tasks.
27	Menu chức năng	Menu	The interface component that lets users access system modules such as ticket sales, Statistical reports, movie/theater management, and membership registration. Provides navigation and reduces errors in user operations.
28	Báo cáo thống kê	Statistical reports	A collection of aggregated data generated from the Database according to selected criteria. These reports include indicators like Revenue, number of sold Tickets, and customer metrics, supporting managerial decision-making.
29	Thống kê phim	Movie statistics	Reports focusing on each Movie, showing tickets sold, Revenue by period, and comparison between films. Helps identify key-performing movies and optimal screening times.
30	Thống kê vé bán	Ticket sales statistics	Reports based on Showtime, indicating the number of Tickets sold in each time slot. Useful for evaluating the performance of specific showtimes and adjusting scheduling strategies.
31	Doanh thu	Revenue	The financial figure representing total income from selling Tickets, categorized by Movie, Showtime, Theater, and time range. Revenue is a key indicator in Statistical reports used for performance evaluation and planning.

2. Project Information:

a. **Objective:**

This desktop-based cinema management application aims to support different users in performing their main functions: management staff can view statistics, manage movies and theaters, and schedule showtimes; saler can issue membership cards to customers; customers can register as members, update personal information, and search for movies and showtimes.

b. **Scope:**

- **Type of application:** Winform (desktop-based application).
- **Users and their corresponding granted functions:** only the cinema's staff (management staff, saler) and customers could use this application.
    + **Management staff:**
        - View the types of statistics: movies, customers and revenue.
        - Manage movie information, theater information (add, edit, delete).
        - Schedule showtimes.
    + **Saler:**
        - Issue membership cards to customers.
        - Sell tickets at the counter to customers.
    + **Customer:**
        - Register for membership.
        - Search tickets online.
        - Buy tickets online.
        - Buy tickets at the counter.

c. **How do the modules work:**

- ***Customer registers as a member***: chooses to register as a member → enters personal information and clicks register → the system saves information to the database.
- ***Management staff views movie statistics***: selects the menu to view statistical reports → chooses to view movie statistics by revenue → selects start and end date → Views movie statistics → clicks on a movie to view details → views statistics of movie showtimes → clicks on a showtime → views statistics of ticket sales of the showtime.

d. **Information about objects:**

- Movie:

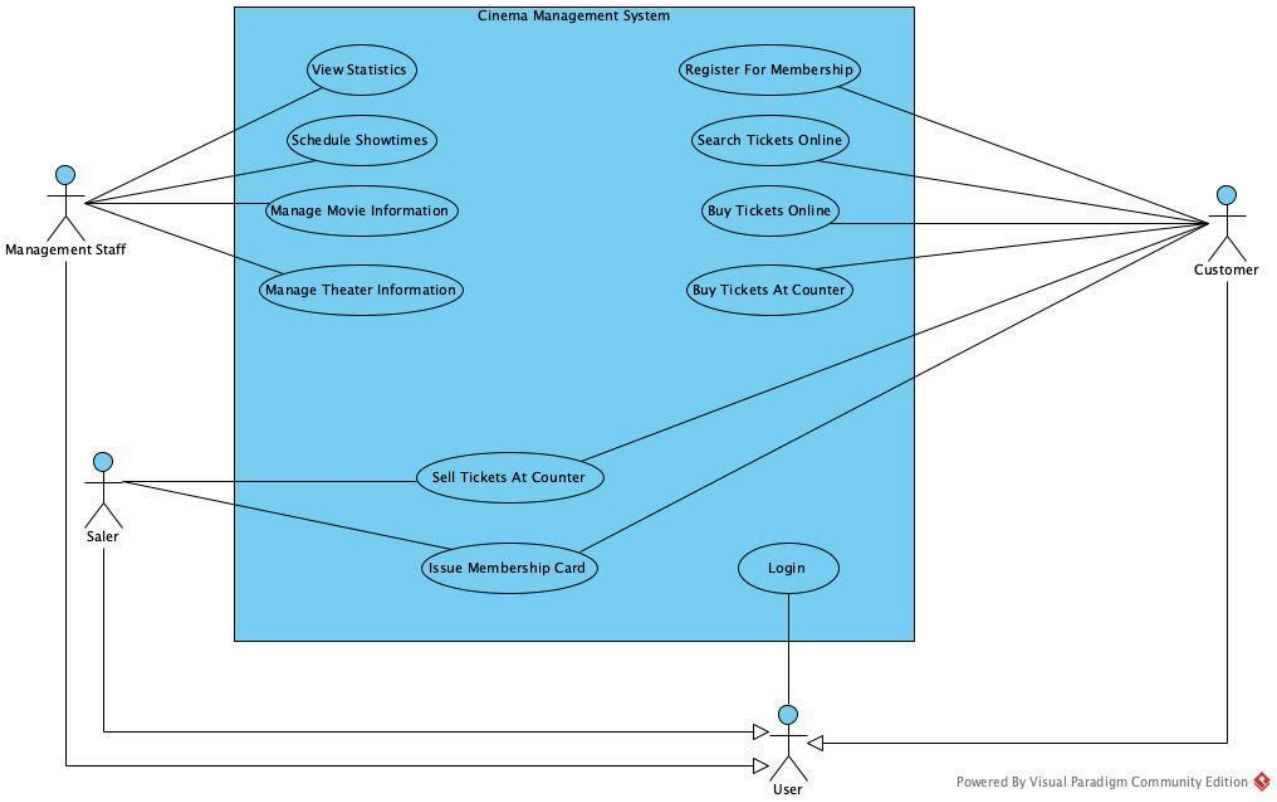
    + Attributes: Movie Name, Movie ID, Duration, Director, Description, Theater.
- Ticket:
    + Attributes: Ticket ID, Movie Name, Show Time, Show Date, Seat, Ticket Price.
- Customer:
    + Attributes: Customer ID, Full Name, Date of Birth, Address, Phone Number, Membership Card Number, Username, Password.
- Management Staff:
    + Attributes: Staff ID, Full Name, Date of Birth, Address, Phone Number, Username, Password.
- Saler:
    + Attributes: Staff ID, Full Name, Date of Birth, Address, Phone Number, Username, Password.
- Showtime:
    + Attributes: Showtime ID, Movie ID, Show Date, Show Time, Room ID, Number of Tickets Sold.
- Theater:
    + Attributes: Room ID, Room Type, Capacity, Status.
- Invoice:
    + Attributes: Invoice ID, Invoice Date, Customer ID, Total Amount.

e. Relationships among objects:

- A theater can have multiple showtimes at different times.
- A theater can only have one showtime at a given time.
- A movie can be shown in multiple theaters at the same time.
- A theater can only show one movie at a given time.
- A showtime is for one specific movie only.
- A ticket is valid for one specific showtime only.
- A showtime can have many tickets sold.
- A theater can have many tickets sold.
- An invoice can include multiple tickets.
- A customer has only one membership card.
- A customer can have multiple invoices.
- A customer can purchase multiple tickets.

3. Draw Use case diagrams:

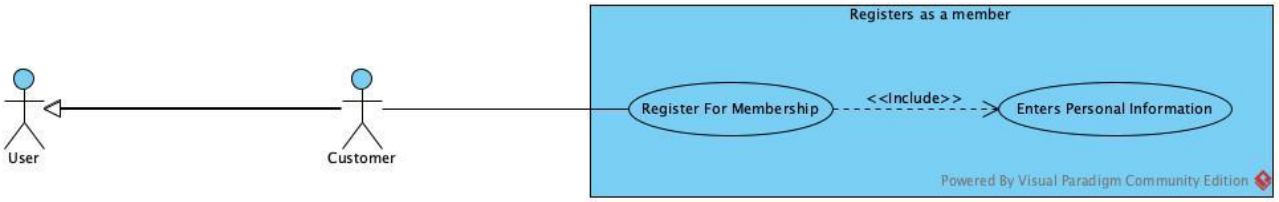
a. **General use case diagram (for the whole system):**



- **Description of UC:**
  - + **Login:** This use case enables users to log in to the cinema management system using their credentials.
  - + **View statistics:** This use case enables management staff to view different types of cinema statistics such as movies, customers, and revenue.
  - + **Schedule showtimes:** This use case enables management staff to create, edit, and manage movie showtimes.
  - + **Manage movie information:** This use case enables management staff to add, update, and delete movie information in the system.
  - + **Manage theater information:** This use case enables management staff to manage theater.
  - + **Sell tickets at counter:** This use case enables the saler to sell tickets directly to customers at the counter.
  - + **Issue Membership Card:** This use case enables the saler to issue membership cards for customers.
  - + **Register for Membership:** This use case enables customers to register for a membership by providing personal information.
  - + **Search Tickets Online:** This use case enables customers to search tickets online.
  - + **Buy Tickets Online:** This use case enables customers to purchase tickets online through the system.
  - + **Buy Tickets at Counter:** This use case enables customers to buy tickets directly at the cinema counter.

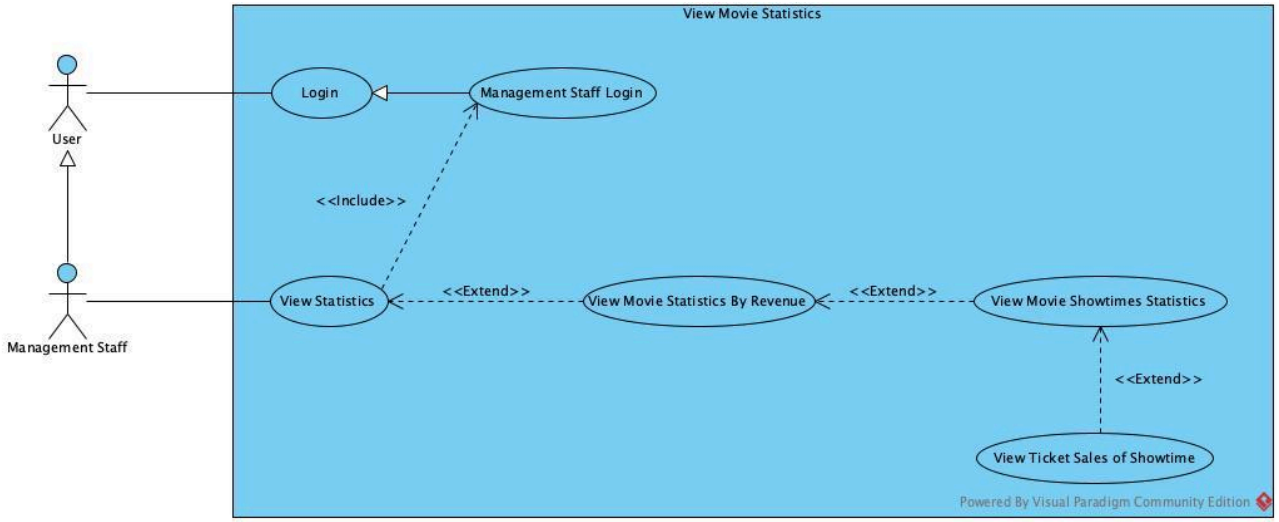
b. Detail use case diagram (for each function):

+ **Module 1: Customer registers as a member:**



- **Description of UC:**
  + **Register for Membership:** This use case enables a user to register as a customer by registering for membership.
  + **Enters personal information:** This use case enables a user to enter personal information as part of the registration process.

+ **Module 2: Management staff views movie statistics:**



- **Description of UC:**
  + **Login:** This use case enables users to authenticate and access the movie statistics system by providing their credentials to gain entry to the system.
  + **Management Staff login:** This use case enables management staff to perform specialized authentication with elevated privileges to access administrative functions and comprehensive statistical views within the movie statistics system.
  + **View Statistics:** This use case enables management staff to access and view general statistical information about movies, providing a central dashboard for various statistical data and reports.
  + **View Movie Statistics by Revenue:** This use case enables management staff to view detailed financial statistics and revenue analysis.
  + **View Movie Showtimes Statistics:** This use case enables management staff to view comprehensive statistics about movie showtimes.
  + **View Ticket Sales of Showtime:** This use case enables management staff to view detailed ticket sales data for specific showtimes.

B. Analysis

1. Scenarios:

a. Customer registers as a member:

Use case	Registers as a member
Actor	Customer
Precondition	The customer does not yet exist in the system.
Postcondition	The customer successfully registers as a member.
Scenario	1. After logging in, from the main interface, the customer selects the Registers for Membership function. 2. The registration interface appears, displaying a customer information form and a Register button. 3. The customer fills in the required information and clicks Register. 4. The system displays a message confirming successful registration.
Exception	The customer is already a member.

b. Management staff views movie statistics:

Use case	Views movie statistics by revenue
Actor	Management staff
Precondition	The management staff has logged into the system with the correct privileges.
Postcondition	- The management staff has successfully viewed the list of movie statistics sorted by revenue in a specified time period. - The management staff is able to click on a movie to view its detailed showtime revenue statistics. - The management staff is able to click on a showtime to view the ticket sales of that showtime.
Scenario	1. After logging in, Management Staff selects the View Statistics function from the main interface. 2. The system displays the Select Statistics Type interface. 3. The Management Staff selects Movie Revenue Statistics, enters the start date 01/01/2024 and end date 09/09/2024, then clicks Statistics. 4. The system displays the Movie Statistics interface. A list of movies appears, sorted in descending order by total revenue:

ID	Movie Name	Total Tickets Sold	Total Revenue (VNĐ)
1	Tấm Cám	10.000	500.000.000
2	Sọ Dừa	5.000	250.000.000

	<table><tr><td>3</td><td>Sơn Tinh và Thuỷ Tinh</td><td>1.000</td><td>50.000</td></tr></table> 5. The Management Staff clicks on the movie "Tấm Cám". 6. The system displays the Showtime Statistics interface for that movie, sorted by showtime in chronological order: <table><tr><td>ID</td><td>Show Time</td><td>Show Date</td><td>Room</td><td>Tickets Sold</td><td>Revenue (VNĐ)</td></tr><tr><td>1</td><td>9:00</td><td>01/01/2025</td><td>A1</td><td>100</td><td>5.000.000</td></tr><tr><td>2</td><td>9:00</td><td>02/01/2025</td><td>A1</td><td>90</td><td>4.500.000</td></tr></table> 7. The Management Staff clicks on the showtime with ID = 1, 9:00 AM, 01/01/2025. 8. The system displays the Invoice Statistics interface showing all invoices related to that showtime: <table><tr><td>ID</td><td>Customer Name</td><td>Total Tickets</td><td>Invoice Amount (VND)</td></tr><tr><td>1</td><td>Nguyễn Minh Khiêm</td><td>2</td><td>100.000</td></tr><tr><td>2</td><td>Nguyễn Hoàng Nguyên</td><td>1</td><td>50.000</td></tr></table> 9. The Management Staff reviews the statistics and clicks Close. 10. The system returns to the Statistics Type Selection interface.
Exception	3. The Management Staff selects Movie Revenue Statistics, enters the start date 01/01/2025 and end date 09/09/2025, then click Statistics. 3.1. The system displays a message: “Invalid time period.” 3.2. The Management Staff clicks OK on the message. 3.3. The system returns to the Select Time Period interface. 3.4. The Management Staff enters the start date 01/01/2024 and end date 09/09/2024, then clicks Statistics again. 3.5. The system displays the list of movie statistics (Step 4).

2. Entity class diagrams:

a. System Description in one paragraph:

The cinema management system supports operations for three types of users: management staff, saler, and customers. Management staff are able to view statistics about movies and customers (based on revenue), schedule showtimes, and maintain data of movies and theaters (add, update, delete). Saler are responsible for selling tickets at the counter, issuing invoices for purchases, and creating membership cards when requested. Customers can register for membership, browse movies and view detailed movie information (title, genre, release date, length, description), and purchase tickets either online or at the counter, with the option to choose movie, theater, showtime, and seat.

b. The extracted nouns:

- Nouns related to people: user, saler, management staff, customer

- Nouns related to objects: cinema, movie, ticket, membership card, theater, showtime, invoice
- Nouns related to information: movie revenue statistics, showtime statistics, invoice statistics

c. Evaluate and Select Nouns as Entity Classes or Attributes:

- Nouns related to people:
  + User → Class User
    - Attributes:
      - ➢ name
      - ➢ username
      - ➢ password
      - ➢ phoneNumber
      - ➢ userType
  + Management Staff → Class Staff (Inherits from User, adds:)
    - position
  + Saler → Class Staff (Inherits from User, adds:)
    - position
  + Customer → Class Customer (Inherits from User, adds:)
    - points
    - membershipCardID
- Nouns related to objects:
  + Movie → Class Movie
    - Attributes:
      - ➢ movieID
      - ➢ movieName
      - ➢ genre
      - ➢ director
      - ➢ duration
  + Theater → Class Theater
    - Attributes:
      - ➢ roomID
      - ➢ roomType
      - ➢ seatCount
  + Showtime → Class Showtime
    - Attributes:
      - ➢ showtimeID
      - ➢ showDate
      - ➢ startTime
      - ➢ endTime
  + Ticket → Class Ticket
    - Attributes:
      - ➢ ticketID

        - ➢ customerName
        - ➢ roomName
        - ➢ showtimeName
        - ➢ seatNumber
        - ➢ price
    + Invoice → Class Invoice
        - Attributes:
            - ➢ invoiceID
            - ➢ customerName
            - ➢ invoiceDate
            - ➢ ticketQuantity
            - ➢ totalAmount
- Nouns related to information:
    + Movie Revenue Statistics → Class MovieStats
    + Showtime Statistics → Class ShowtimeStats
    + Invoice Statistics → Class Invoice (same class name because statistics are based on invoices)

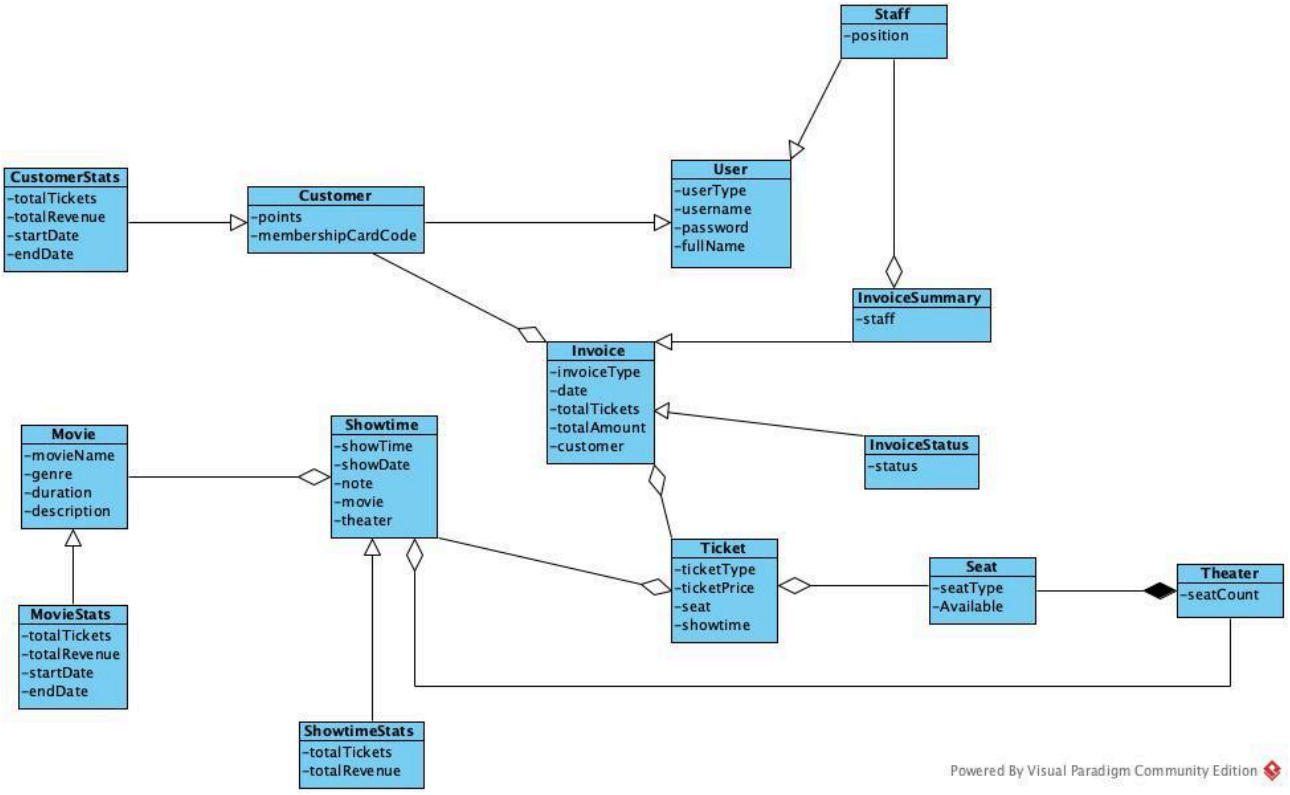
d. Determine Quantitative Relationships Between Entities:

- A theater can have multiple showtimes at different times → Theater – Showtime (1–n)
- A theater can only have one showtime at a given time → Theater – Showtime (1–1 at a specific time slot) (Constraint: unique time slot per theater)
- A movie can be shown in multiple theaters at the same time → Movie – Theater (1–n) (per time slot)
- A theater can only show one movie at a given time → Theater – Movie (1–1 at a specific time slot)
- A showtime is for one specific movie only → Showtime – Movie (n–1)
- A ticket is valid for one specific showtime only → Ticket – Showtime (n–1)
- A showtime can have many tickets sold → Showtime – Ticket (1–n)
- A theater can have many tickets sold → Theater – Ticket (1–n)
- An invoice can include multiple tickets → Invoice – Ticket (1–n) (Each ticket belongs to exactly one invoice → Ticket – Invoice (n–1))
- A customer has only one membership card → Customer – MembershipCard (1–1)
- A customer can have multiple invoices → Customer – Invoice (1–n)
- A customer can purchase multiple tickets → Customer – Ticket (1–n)

e. Determine Object Relationships Between Entities:

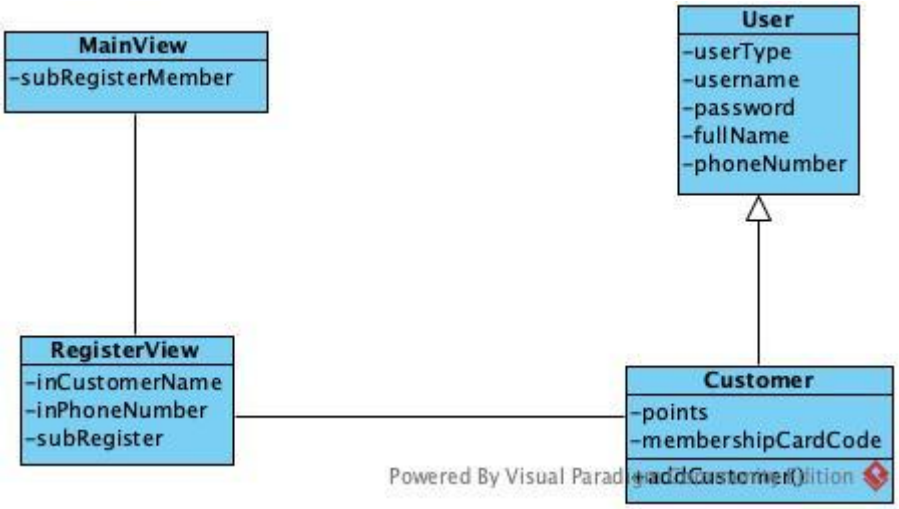
- Inheritance Relationships
    + Staff inherits from User
    + Management Staff and Saler inherit from Staff

    + Customer inherits from User
    + CustomerStats inherits (generalizes) Customer
    + MovieStats inherits (generalizes) Movie
- Association Relationships (1–n)
    + Cinema – Theater (1–n)
    + Theater – Seat (1–n)
    + Invoice – Ticket (1–n)
    + Customer – Invoice (1–n)
- Statistics Relationships (Generalization 1–1)
    + MovieStats – Movie (1–1)
    + CustomerStats – Customer (1–1)
- One-to-One (1–1) Relationships
    + Customer – MembershipCard (1–1, optional)
    + Ticket – ShowtimeSeat (1–1)



3. Extract and draw the class diagram for the module:

a. Module 1: Customer registers as a member:



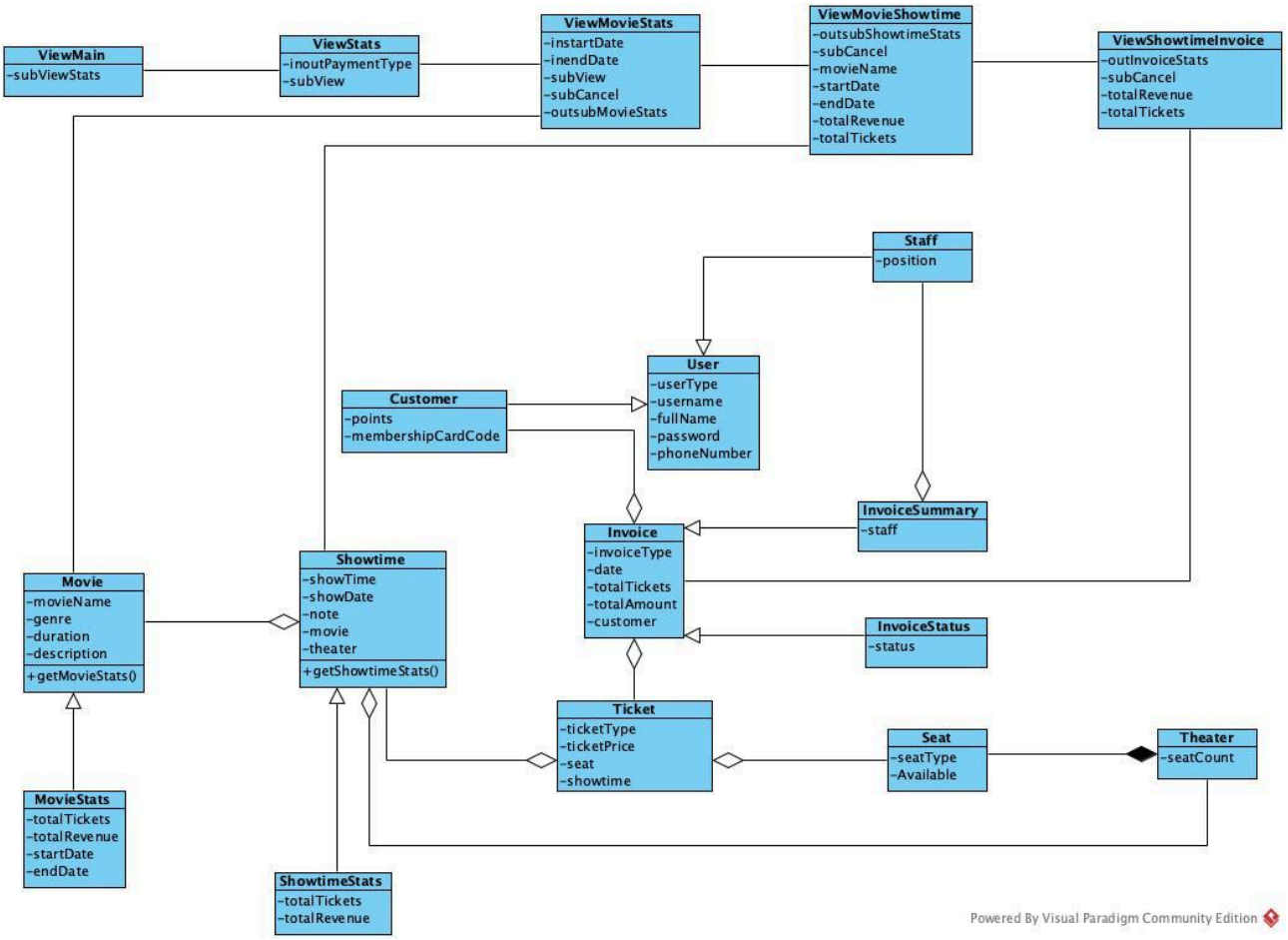
- The customer enters the system → The main interface appears → Needs class MainView, which has:
    + A button to register member → subRegisterMember
- The customer chooses RegisterMember → The registration interface appears → Needs a class: RegisterView
    + Input for customer name → inCustomerName
    + Input for phone number → inPhoneNumber
    + A button to register → subRegister

  Class: User (Entity)
    - Attributes:
        + userType
        + username
        + password
        + fullName
        + phoneNumber

  Class: Customer (Entity) - inherits from User
    - Attributes:
        + points
        + membershipCardCode
    - Operation:
        + addCustomer() : boolean
- The customer enters name and phone number in RegisterView, clicks subRegister → The system must save the information to the database → Needs a method:
    + Name: AddCustomer()
    + Input: fullName, phoneNumber
    + Output: boolean (success / failure)

  This method is assigned to the entity class Customer, which inherits from User.

b. Module 2: Management staff views movie statistics:



- The management staff enters the system → The main management interface appears → Needs class ViewMain, which has:
    + A button to view stats → subViewStats
- The management staff chooses Stats → The stats menu interface appears → Needs class ViewStats, which has:
    + Input, output for payment type → inoutPaymentType
    + A sub view button → subView
- The management staff chooses View Movie Stats → The movie statistics interface appears → Needs class ViewMovieStats, which has:
    + Input for start date → instartDate
    + Input for end date → inendDate
    + A sub view button → subView
    + A sub cancel button → subCancel
    + Output movie stats → outsubMovieStats
- The management staff selects a movie → The movie showtime interface appears → Needs class ViewMovieShowtime, which has:
    + Output showtime stats → outsubShowtimeStats
    + A sub cancel button → subCancel
    + Movie name → movieName
    + Start date → startDate
    + End date → endDate

    + Total revenue → totalRevenue
    + Total tickets → totalTickets
- The management staff chooses to view showtime invoices → The showtime invoice interface appears → Needs class ViewShowtimeInvoice, which has:
        + Output invoice stats → outInvoiceStats
        + A sub cancel button → subCancel
        + Total revenue → totalRevenue
        + Total tickets → totalTickets

Entity Classes:

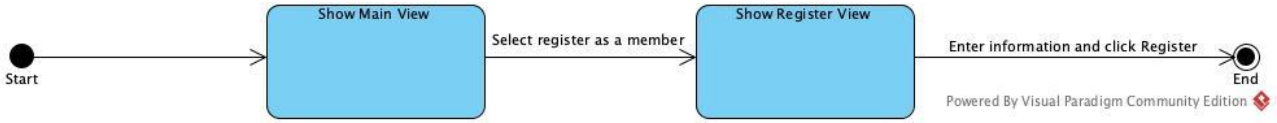
- Movie (Entity)
    + Attributes: movieName, genre, duration, description
    + Operation: getMovieStats()
- MovieStats (Entity - inherits from Movie)
    + Attributes: totalTickets, totalRevenue, startDate, endDate
- Showtime (Entity)
    + Attributes: showTime, showDate, note, movie, theater
    + Operation: getShowtimeStats()
- ShowtimeStats (Entity - inherits from Showtime)
    + Attributes: totalTickets, totalRevenue
- Theater (Entity)
    + Attributes: seatCount
- Seat (Entity)
    + Attributes: seatType, Available
- Ticket (Entity)
    + Attributes: ticketType, ticketPrice, seat, showtime
- Invoice (Entity)
    + Attributes: invoiceType, date, totalTickets, totalAmount, customer
- InvoiceSummary (Entity)
    + Attributes: staff
- InvoiceStatus (Entity)
    + Attributes: status
- User (Entity)
    + Attributes: userType, username, fullName, password, phoneNumber
- Staff (Entity - inherits from User)
    + Attributes: position
- Customer (Entity - inherits from User)
    + Attributes: points, membershipCardCode

Method Assignments:

- **getMovieStats()**:

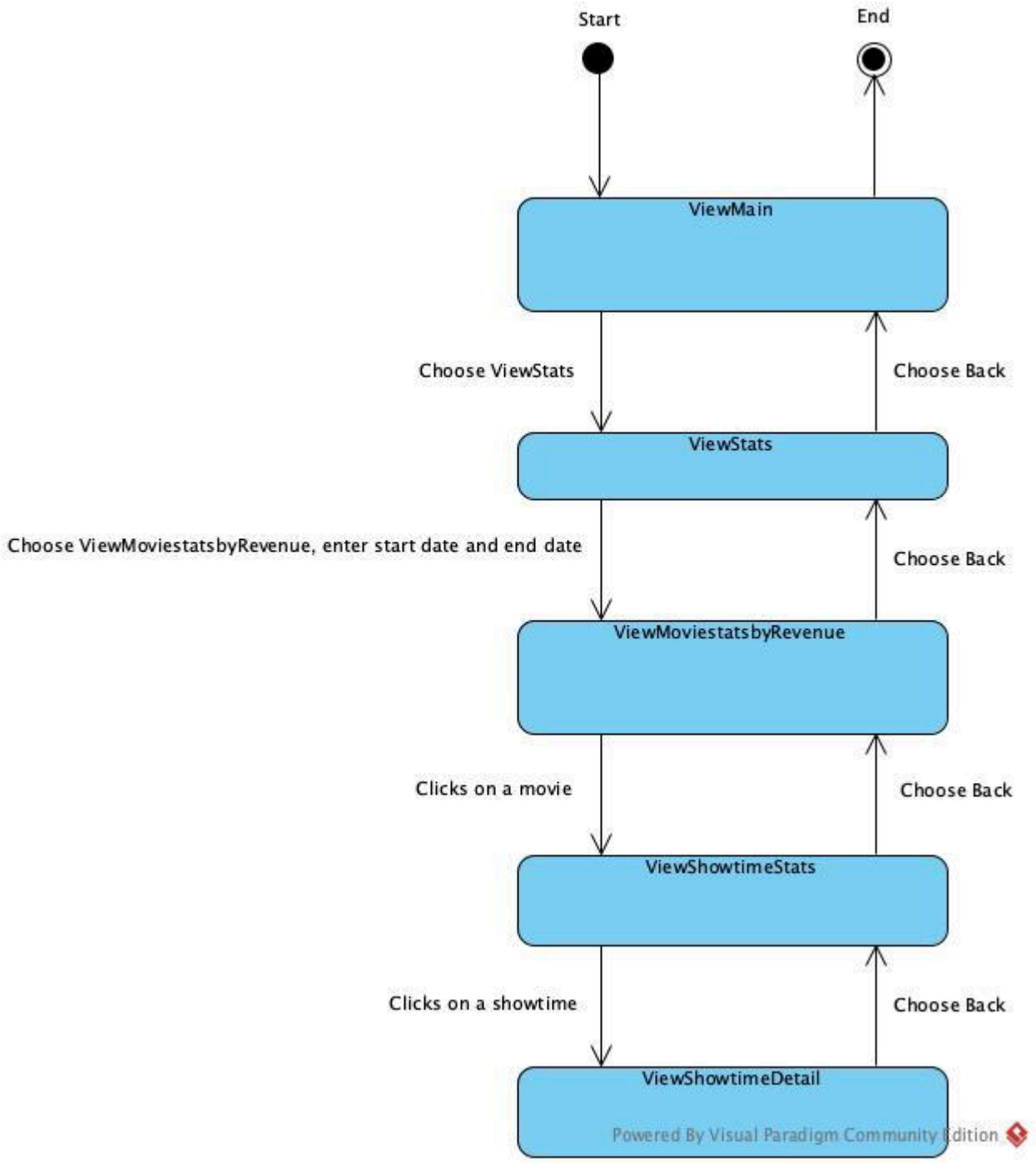
    + Input: startDate, endDate, paymentType
    + Output: List<MovieStats>
    + Assigned to Movie class
- **getShowtimeStats()**:
    + Input: movieName, startDate, endDate
    + Output: ShowtimeStats
    + Assigned to Showtime class

4. **Draw the state diagram for the module:**
    a. **Module 1: Customer registers as a member:**



- From the Show Main View: When the system first starts, it displays the Show Main View. At this stage, the customer is presented with a choice. If the customer selects "Select register as a member", the system transitions to the Show Register View, where the registration process can begin.
- From the Show Register View: Once the Show Register View is displayed, the customer is able to "Enter information and click Register". When the customer completes this action, the system saves the provided registration details and the process reaches the End state successfully.
- End State: The process reaches the end state when the customer completes the registration by entering information and clicking Register from the Show Register View. At this point, the system concludes the process and no further transitions take place.

    b. **Module 2: Management staff views movie statistics:**



- From the Start: The system begins at the ViewMain interface.
- From the ViewMain interface: If the manager chooses ViewStats, the system navigates to the ViewStats interface. If the manager chooses Back, the system returns to the End state.
- From the ViewStats interface: If the manager chooses ViewMoviestatsbyRevenue, enter start date and end date, the system navigates to the ViewMoviestatsbyRevenue interface. If the manager chooses Back, the system returns to the ViewMain interface.
- From the ViewMoviestatsbyRevenue interface: If the manager clicks on a movie, the system navigates to the ViewShowtimeStats interface. If the manager chooses Back, the system returns to the ViewStats interface.

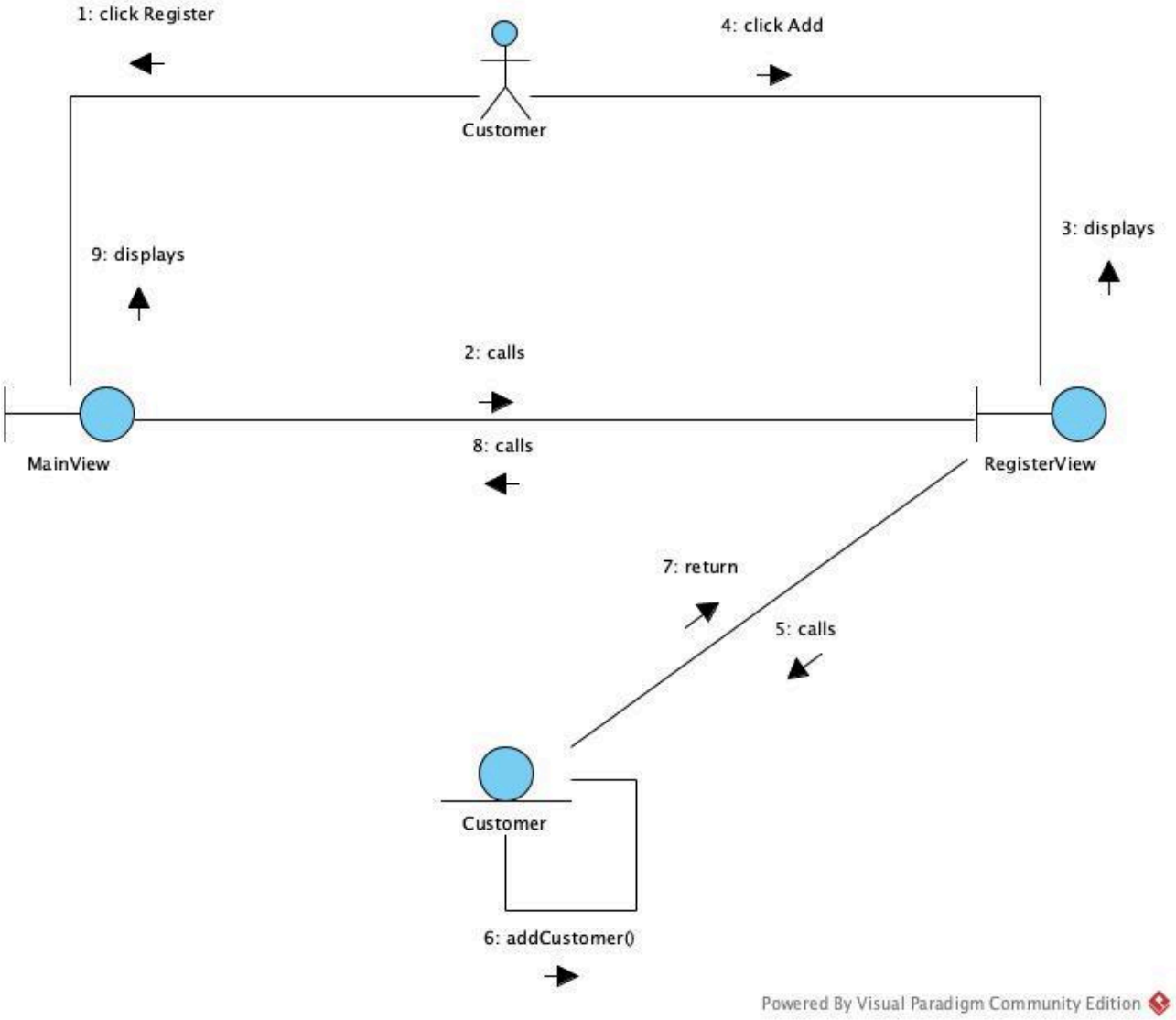
- From the ViewShowtimeStats interface: If the manager clicks on a showtime, the system navigates to the ViewShowtimeDetail interface. If the manager chooses Back, the system returns to the ViewMoviestatsbyRevenue interface.
- From the ViewShowtimeDetail interface: The manager can view the showtime details. There is no Back option shown from this interface in the diagram.
- End State: The process reaches the end state when the manager chooses Back from the ViewMain interface.

5. **Write the detailed scenario (ver 2.0):**
    a. **Module 1: Customer registers as a member:**
        1. In the MainView interface, the customer clicks Register.
        2. The MainView calls the RegisterView interface.
        3. The RegisterView interface is displayed to the customer.
        4. In the RegisterView, the customer clicks Add to submit registration information.
        5. The RegisterView calls the Customer class.
        6. The Customer class executes addCustomer() to add a new customer.
        7. The Customer class returns the result of the operation.
        8. The RegisterView calls the MainView interface.
        9. The MainView interface is displayed to the customer.
    b. **Module 2: Management staff views movie statistics:**
        1. On the ViewMain interface, the management staff chooses ViewStats.
        2. ViewMain calls the ViewStats interface.
        3. The ViewStats interface is displayed to the management staff.
        4. The management staff selects Stats Type.
        5. ViewStats calls the ViewMovieStats interface.
        6. ViewMovieStats calls the MovieStats class.
        7. The MovieStats class executes getMovieStats().
        8. MovieStats returns the data.
        9. The ViewMovieStats interface displays the data to the management staff.
        10. The management staff selects a movie.
        11. ViewMovieStats calls the ViewShowtimeStats interface.
        12. ViewShowtimeStats calls the ShowtimeStats class.
        13. The ShowtimeStats class executes getShowtimeStats().
        14. ShowtimeStats returns the showtime statistics.
        15. The ViewShowtimeStats interface displays the data to the management staff.
        16. The management staff selects on showtime.
        17. ViewShowtimeStats calls the ViewShowtimeInvoice interface.
        18. ViewShowtimeInvoice calls the Invoice class.
        19. The Invoice class executes getInvoice().

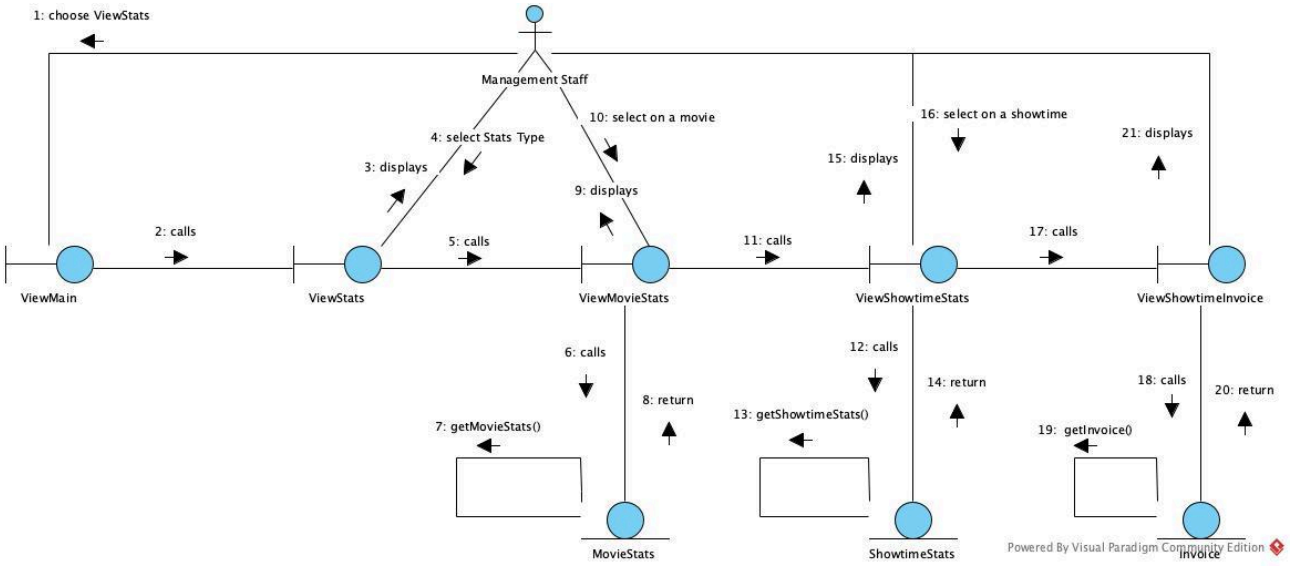
20. Invoice returns the invoice data.
21. The ViewShowtimeInvoice interface displays the data to the management staff.

6. Draw the communication diagram for the module:

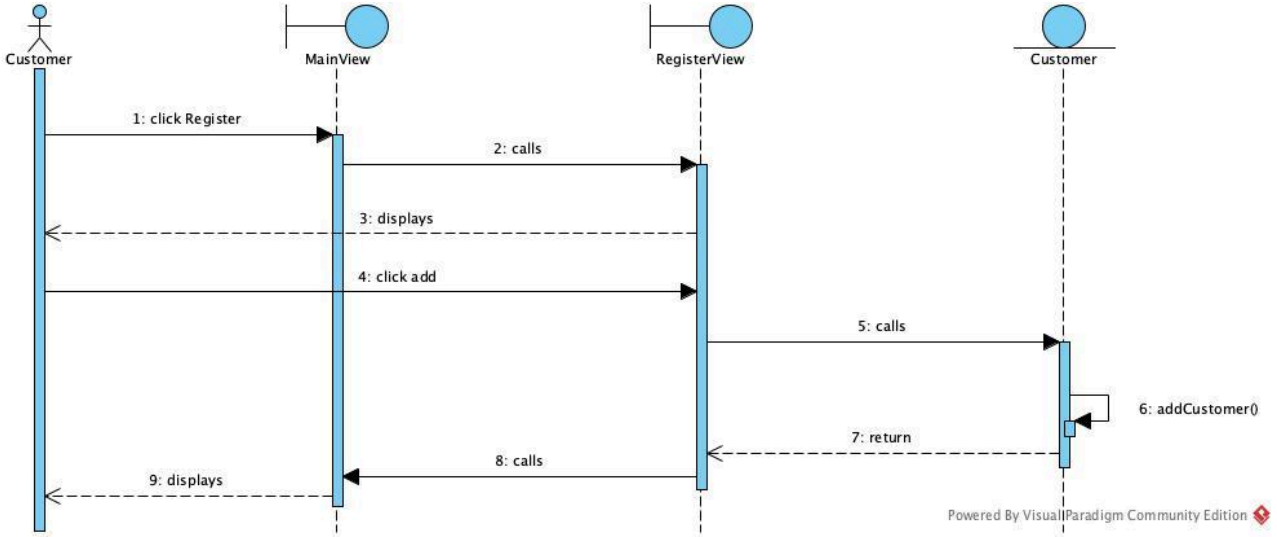
a. Module 1: Customer registers as a member:



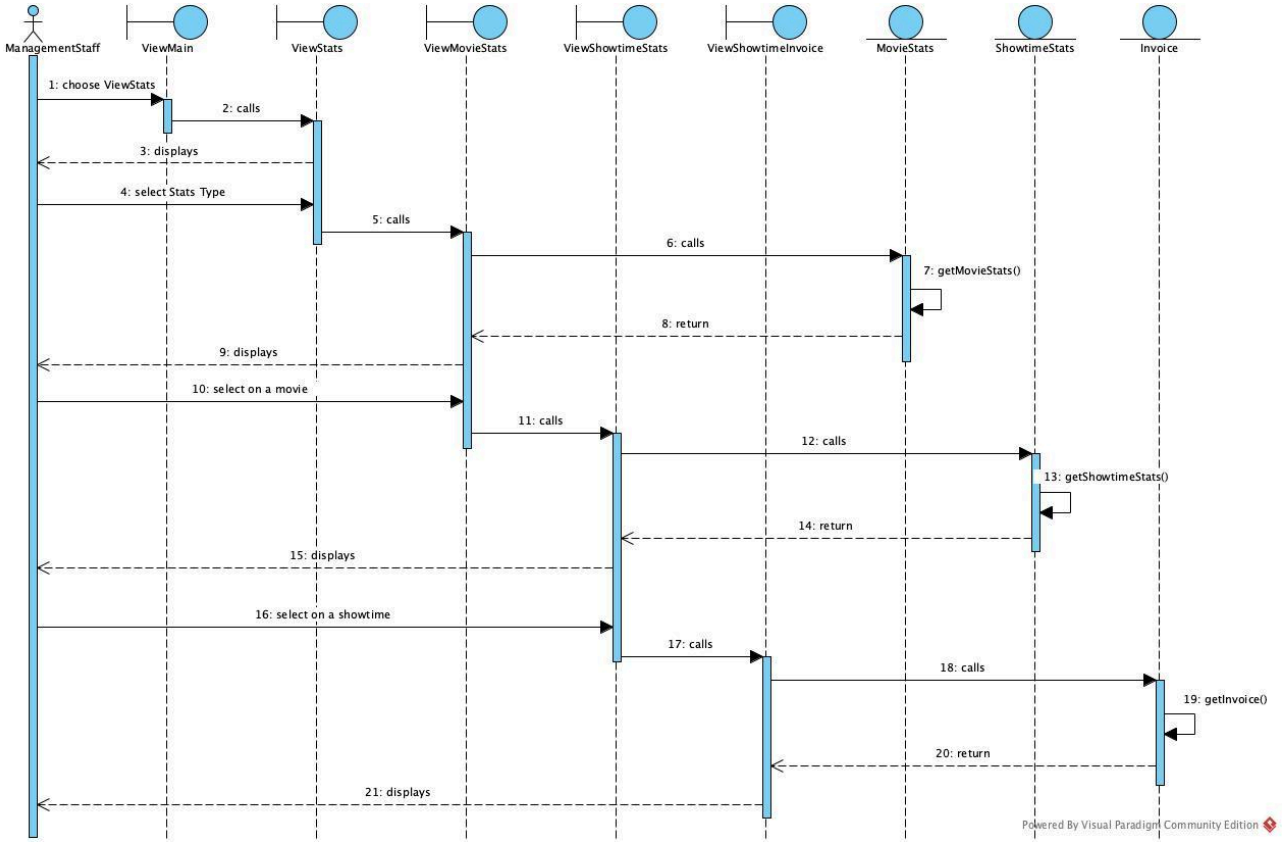
b. Module 2: Management staff views movie statistics:



7. **Draw the sequence diagram for the module based on the communication diagram:**
    a. **Module 1: Customer registers as a member:**

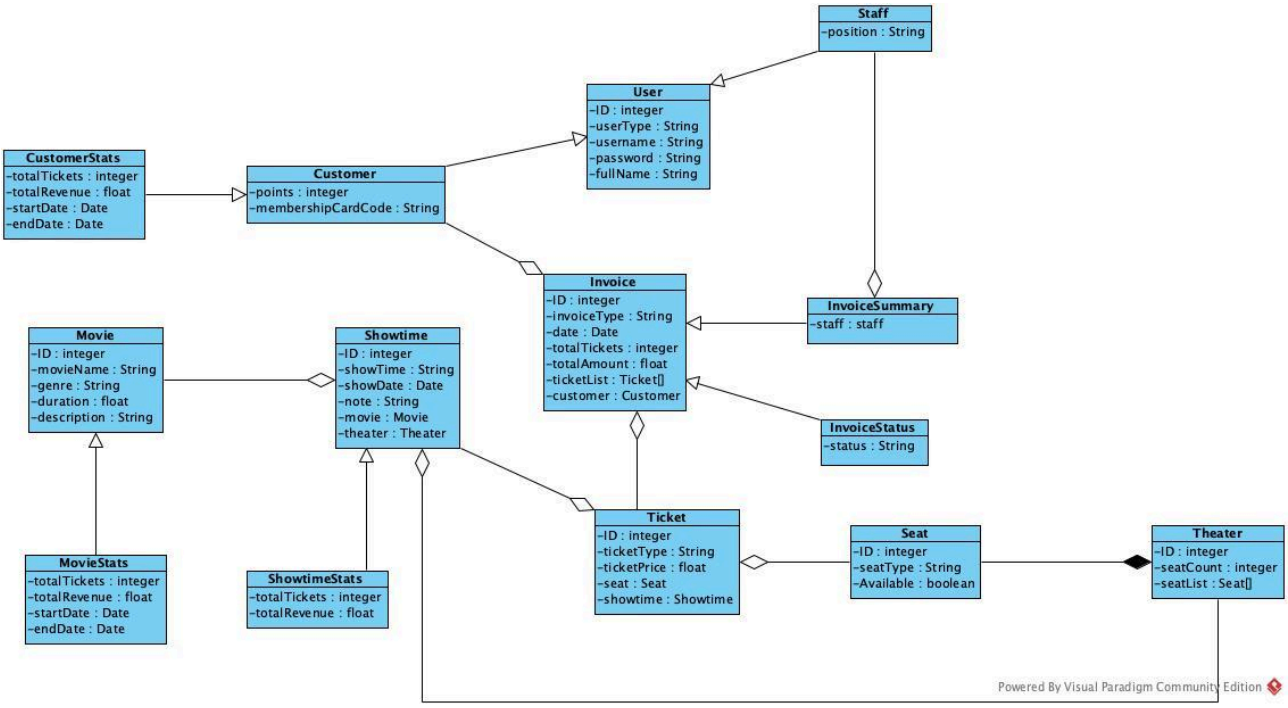


    b. **Module 2: Management staff views movie statistics:**



C. Design

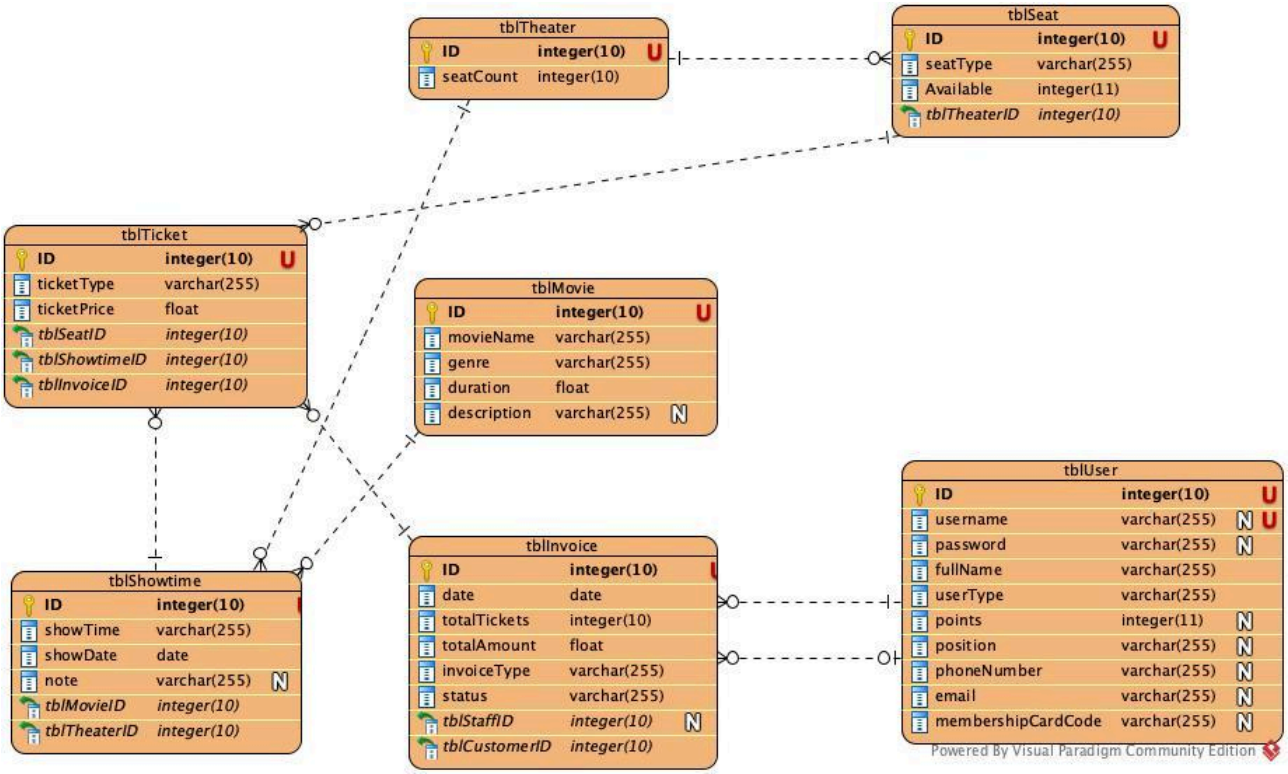
1. Entity class diagram (design phase):



- Step 1: Add id attribute for User, Customer, Staff, Movie, MovieStats, Showtime, ShowtimeStats, Theater, Seat, Ticket, Invoice, InvoiceStatus, InvoiceSummary, CustomerStats
- Step 2: The attributes are added according to Java programming language

- Step 3:
    + Relationship Movie – Showtime changes to Showtime contains Movie
    + Relationship Showtime – Theater changes to Showtime contains Theater
    + Relationship Ticket – Seat changes to Ticket contains Seat
    + Relationship Ticket – Showtime changes to Ticket contains Showtime
    + Relationship Invoice – Customer changes to Invoice contains Customer
    + Relationship Invoice – Ticket becomes ticketList in Invoice
    + Relationship Theater – Seat becomes seatList in Theater
    + Movie has inheritance relationship with MovieStats (MovieStats inherits from Movie)
    + Showtime has inheritance relationship with ShowtimeStats (ShowtimeStats inherits from Showtime)
    + User has inheritance relationship with Customer and Staff (Customer and Staff inherit from User)
- Step 4: Add objects for the entity classes

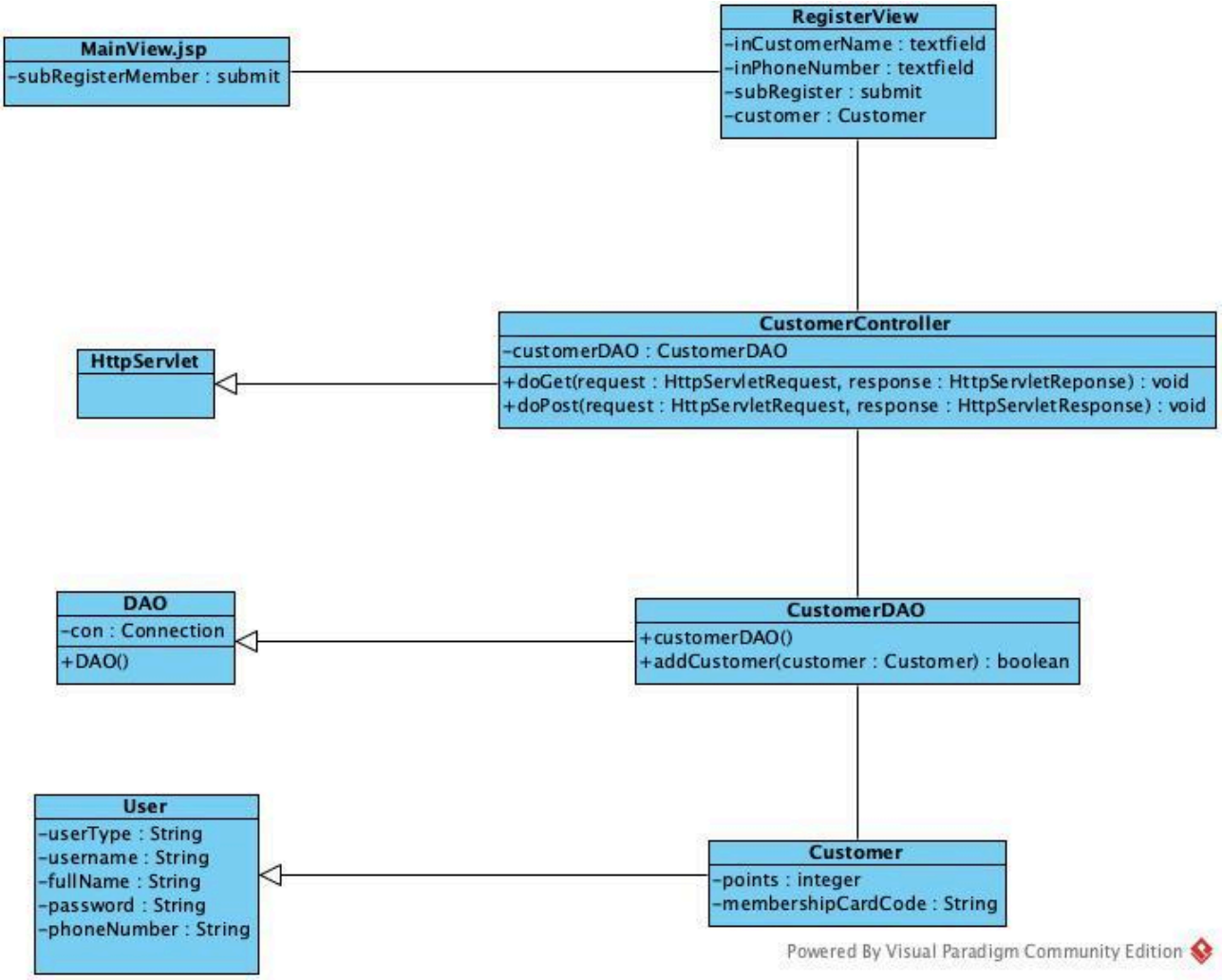
2. Database design:



- **Step 1: For each entity class, create a corresponding table:**
    + Class User -> table tblUser
    + Class Invoice -> table tblInvoice
    + Class Ticket -> table tblTicket
    + Class Theater -> table tblTheater
    + Class Seat -> table tblSeat

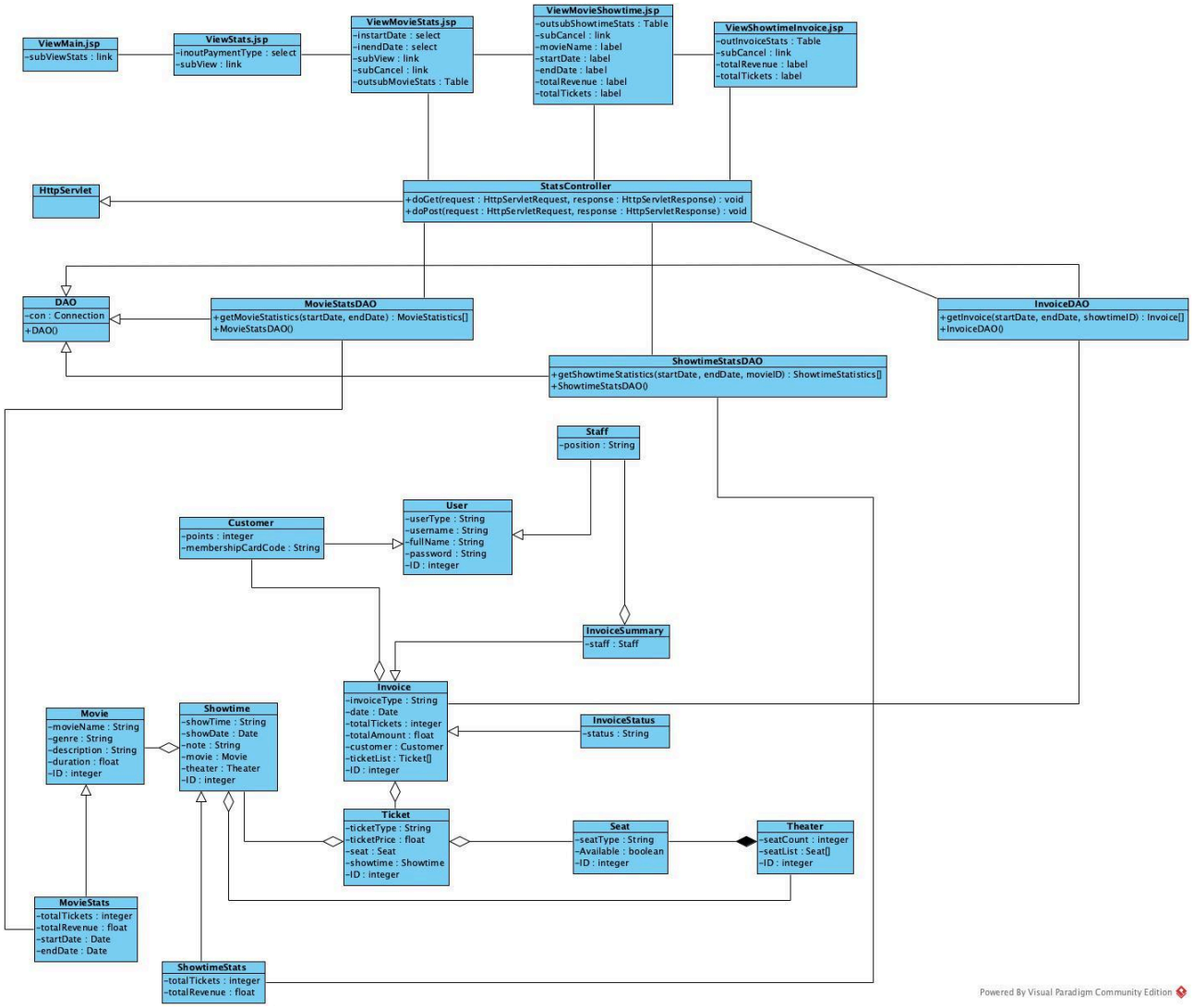
    + Class Showtime -> table tblShowtime
    + Class Movie -> table tblMovie
- **Step 2: For each entity class, transfer all NON-OBJECT attributes to contribute as the columns of the corresponding table:**
    + tblUser: ID, username, password, fullName, userType, points, position, phoneNumber, email, membershipCardCode
    + tblTheater: ID, seatCount
    + tblSeat: ID, seatType, Available
    + tblMovie: ID, movieName, genre, duration, description
    + tblShowtime: ID, showTime, showDate, note
    + tblTicket: ID, ticketType, ticketPrice
    + tblInvoice: ID, date, totalTickets, totalAmount, invoiceType, status
- **Step 3: Consider the quantity relationships among entity classes:**
    + 1 tblTheater - 1..n tblSeat
    + 1 tblMovie - 0..n tblShowtime
    + 1 tblTheater - 0..n tblShowtime
    + 1 tblSeat - 0..n tblTicket
    + 1 tblShowtime - 0..n tblTicket
    + 1 tblInvoice - 1..n tblTicket
    + 1 tblUser - 0..n tblInvoice (as Staff via tblStaffID)
    + 1 tblUser - 0..n tblInvoice (as Customer via tblCustomerID)
- **Step 4: Configure the key columns for tables:**
    + 1 tblTheater - 1..n tblSeat -> tblSeat has foreign key tblTheaterID
    + 1 tblMovie - 0..n tblShowtime -> tblShowtime has foreign key tblMovieID
    + 1 tblTheater - 0..n tblShowtime -> tblShowtime has foreign key tblTheaterID
    + 1 tblSeat - 0..n tblTicket -> tblTicket has foreign key tblSeatID
    + 1 tblShowtime - 0..n tblTicket -> tblTicket has foreign key tblShowtimeID
    + 1 tblInvoice - 1..n tblTicket -> tblTicket has foreign key tblInvoiceID
    + 1 tblUser - 0..n tblInvoice -> tblInvoice has foreign key tblStaffID
    + 1 tblUser - 0..n tblInvoice -> tblInvoice has foreign key tblCustomerID
- **Step 5: Remove attributes creating duplications:**
    + Combined Staff and Customer into tblUser with userType, points, and position attributes to handle both roles
    + Removed separate inheritance tables for better normalization

3. **Class diagrams:**
    a. **Module 1: Customer registers as a member:**



- View layer has the following jsp pages: MainView.jsp, RegisterView
- Controller layer has the following Controller classes: CustomerController, HttpServlet
- Data access layer has the following DAO classes: CustomerDAO, DAO
- Model layer has the following entity classes: User, Customer

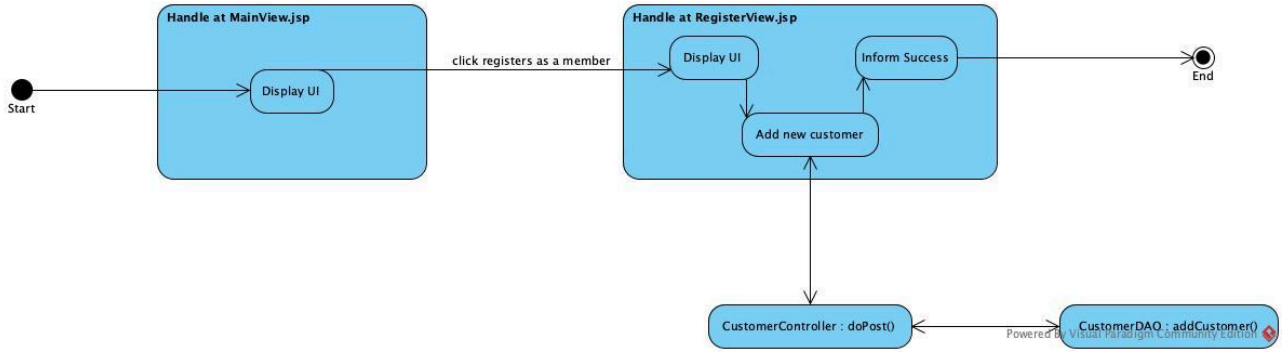
b. Module 2: Management staff views movie statistics:



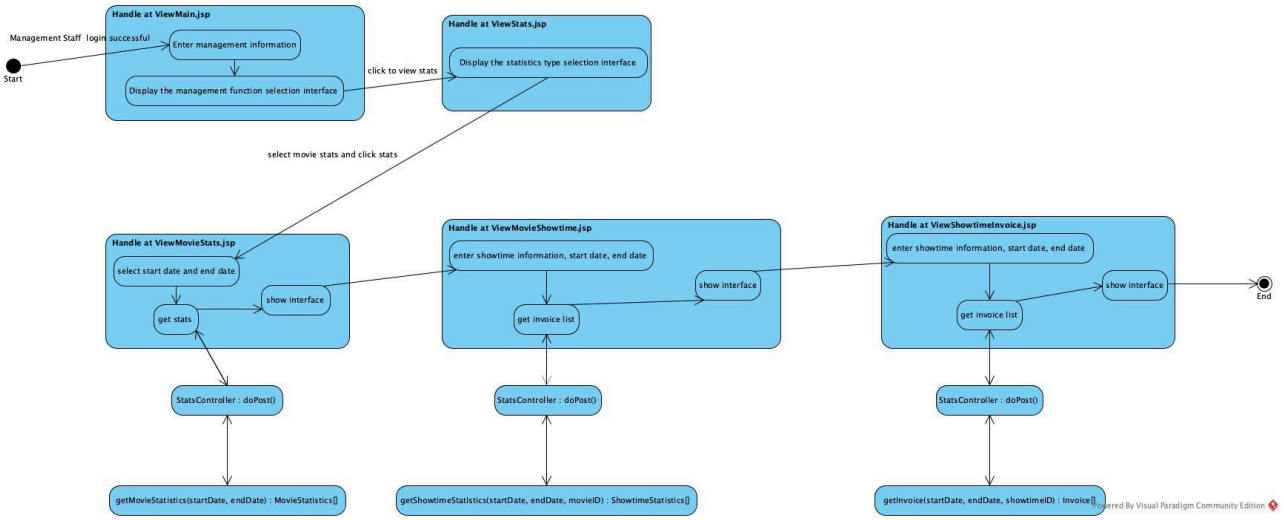
- View layer has the following jsp pages: ViewMain, ViewStats, ViewMovieStats, ViewMovieShowtime, ViewShowtimeInvoice
- Controller layer has the following Controller classes: StatsController, HttpServlet
- Data access layer has the following DAO classes: MovieStatsDAO, ShowtimeStatsDAO, InvoiceDAO, DAO
- Model layer has the following entity classes: Movie, MovieStats, Showtime, ShowtimeStats, Theater, Seat, Ticket, Invoice, InvoiceStatus, InvoiceSummary, User, Customer, Staff

4. Activity diagrams:

a. Module 1: Customer registers as a member:



b. Module 2: Management staff views movie statistics:



5. Scenario 3.0:

a. Module 1: Customer registers as a member:

1. The customer clicks Register on the interface.
2. The page MainView.jsp calls the page RegisterView.jsp.
3. The page RegisterView.jsp displays to the customer.
4. The customer enters and clicks add.
5. The page RegisterView.jsp calls the class CustomerController.
6. The class CustomerController calls the method doPost().
7. The method doPost() calls the class Customer.
8. The class Customer creates a Customer() object.
9. The class Customer returns the Customer object to the method doPost().
10. The method doPost() calls the class CustomerDAO.
11. The class CustomerDAO calls the method addCustomer().
12. The method addCustomer() returns the result to the method doPost().
13. The method doPost() returns the result to the page RegisterView.jsp.
14. The page RegisterView.jsp displays success to the customer.
15. The customer clicks OK.
16. The page RegisterView.jsp calls the page MainView.jsp.
17. The page MainView.jsp displays to the customer.

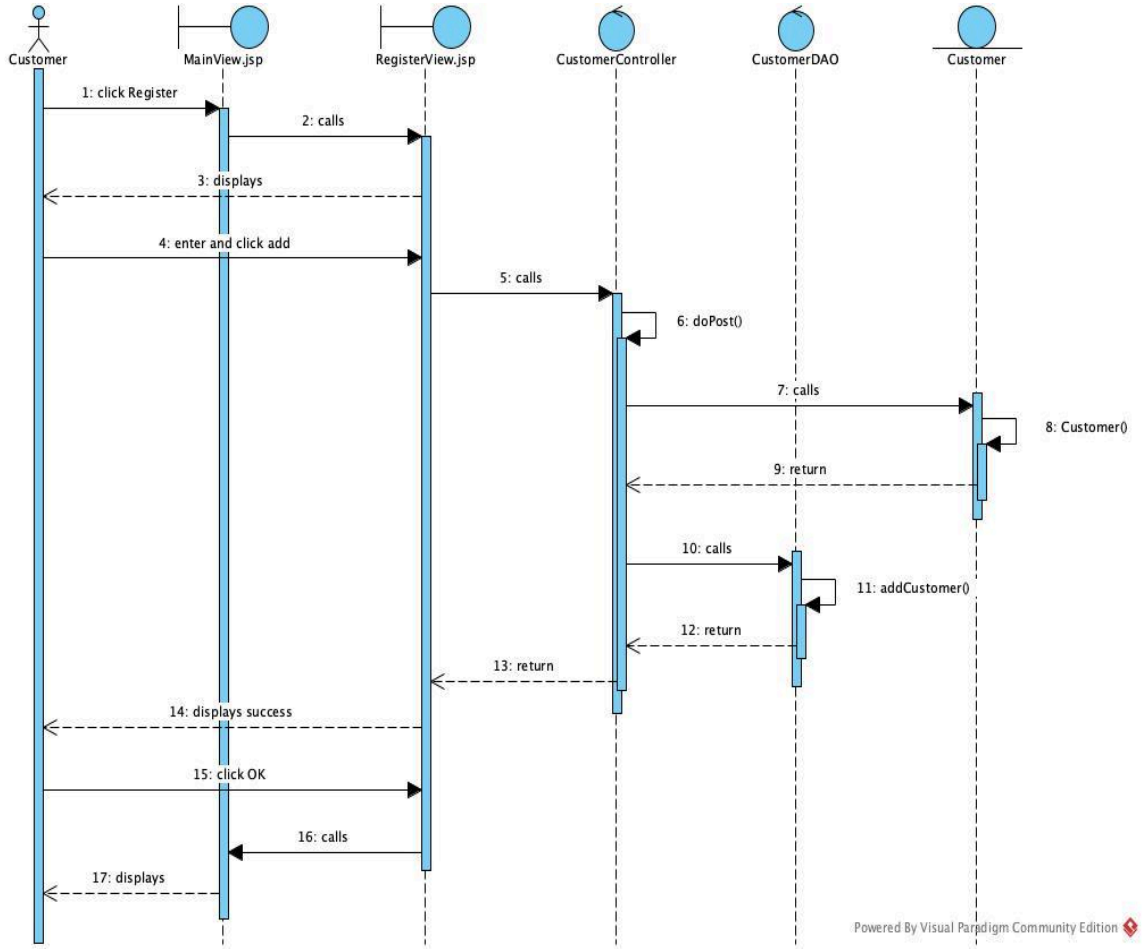
b. Module 2: Management staff views movie statistics:

1. The management staff selects statistics.
2. The page ViewMain.jsp calls the page ViewStats.jsp.
3. The page ViewStats.jsp displays to the management staff.
4. The management staff selects movie statistics by revenue.
5. The page ViewStats.jsp calls the page ViewMovieStats.jsp.
6. The page ViewMovieStats.jsp displays to the management staff.
7. The management staff selects start date, end date and clicks stats.
8. The page ViewMovieStats.jsp calls the class StatsController.
9. The class StatsController calls the method doGet().
10. The method doGet() calls the class MovieStatsDAO.
11. The class MovieStatsDAO calls the method MovieStats().
12. The method MovieStats() returns the results to the method doGet().
13. The method doGet() calls the class MovieStatsDAO.
14. The class MovieStatsDAO calls the method getMovieStats().
15. The method getMovieStats() returns the results to the method doGet().
16. The method doGet() returns the results to the page ViewMovieStats.jsp.
17. The page ViewMovieStats.jsp displays the results to the management staff.
18. The management staff selects a movie.
19. The page ViewMovieStats.jsp calls the page ViewMovieShowtime.jsp.
20. The page ViewMovieShowtime.jsp calls the class StatsController.
21. The class StatsController calls the method doGet().
22. The method doGet() calls the class ShowtimeStatsDAO.
23. The class ShowtimeStatsDAO calls the method ShowtimeStats().
24. The method ShowtimeStats() returns the results to the method doGet().
25. The method doGet() calls the class ShowtimeStatsDAO.
26. The class ShowtimeStatsDAO calls the method getShowtimeStats().
27. The method getShowtimeStats() returns the results to the method doGet().
28. The method doGet() returns the results to the page ViewMovieShowtime.jsp.
29. The page ViewMovieShowtime.jsp displays the results to the management staff.
30. The management staff selects on showtime.
31. The page ViewMovieShowtime.jsp calls the page ViewShowtimeInvoice.jsp.
32. The page ViewShowtimeInvoice.jsp calls the class StatsController.
33. The class StatsController calls the method doGet().
34. The method doGet() calls the class InvoiceDAO.
35. The class InvoiceDAO calls the method Invoice().
36. The method Invoice() returns the results to the method doGet().

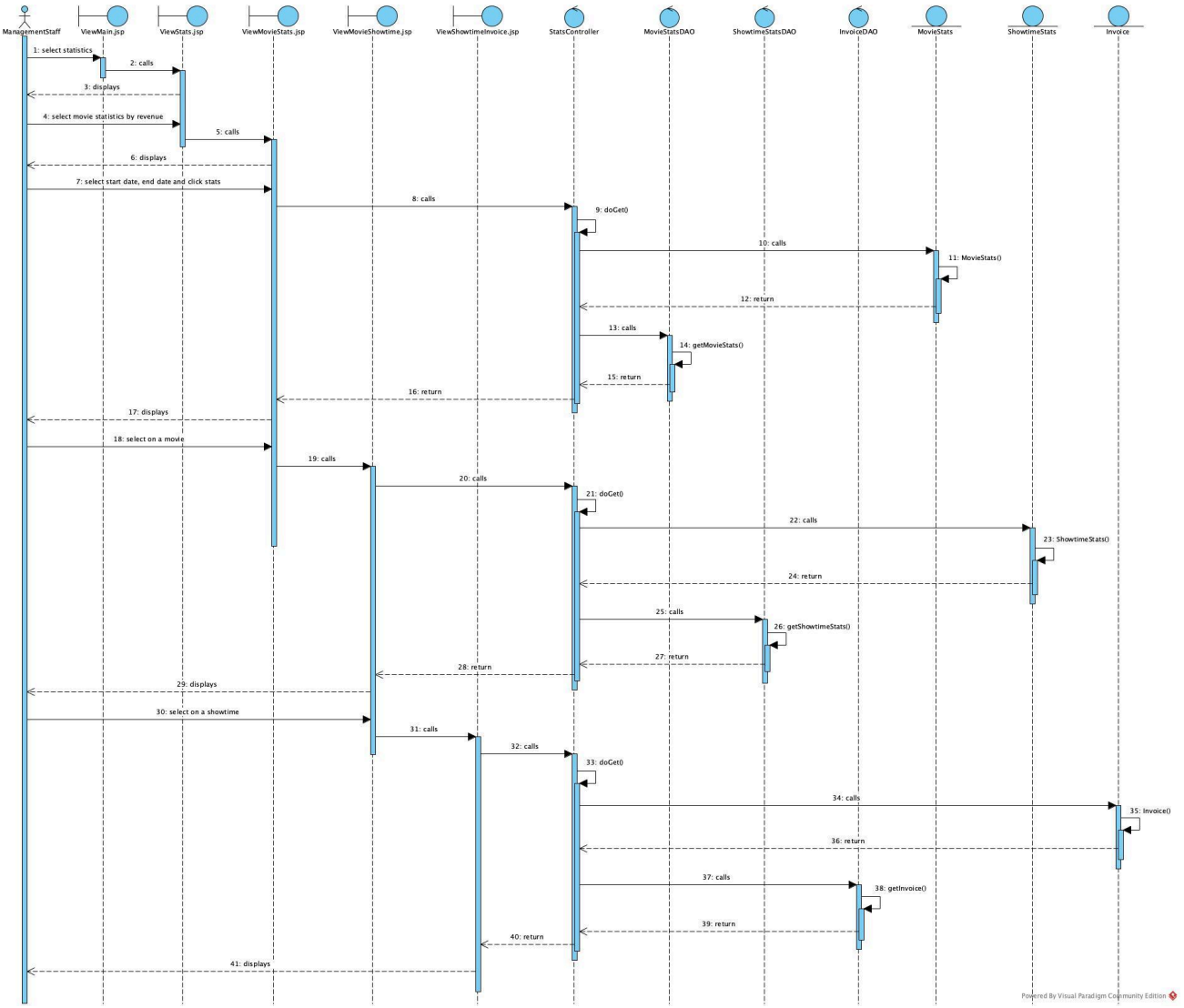
37. The method doGet() calls the class InvoiceDAO.
38. The class InvoiceDAO calls the method getInvoice().
39. The method getInvoice() returns the results to the method doGet().
40. The method doGet() returns the results to the page ViewShowtimeInvoice.jsp.
41. The page ViewShowtimeInvoice.jsp displays the results to the management staff.

6. Sequence diagrams:

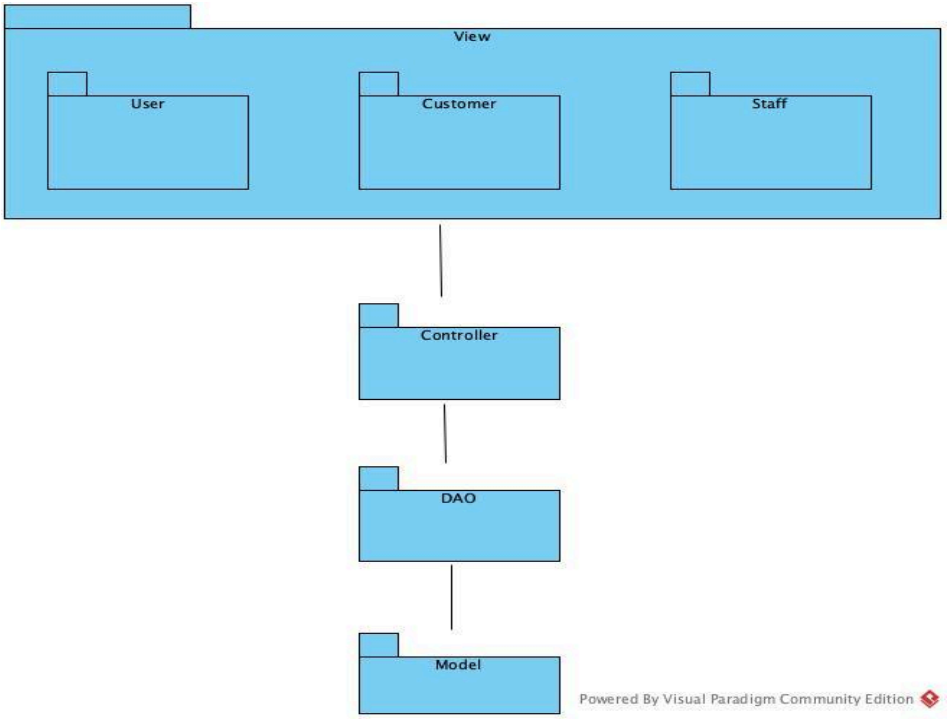
a. Module 1: Customer registers as a member:



b. Module 2: Management staff views movie statistics:



7. Packaged Diagram:



- The packages are designed as follows:
    + Entity classes are placed together in the Model package
    + DAO classes are placed together in the DAO package
    + Controller classes are placed in the Controller package
    + JSP pages are placed in the View package. The View package is divided into smaller packages corresponding to interfaces for different users:
        - Pages for functions related to users are placed in the User package
        - Pages for functions related to customers are placed in the Customer package
        - Pages for functions related to staff are placed in the Staff package

8. Deployment Diagram: